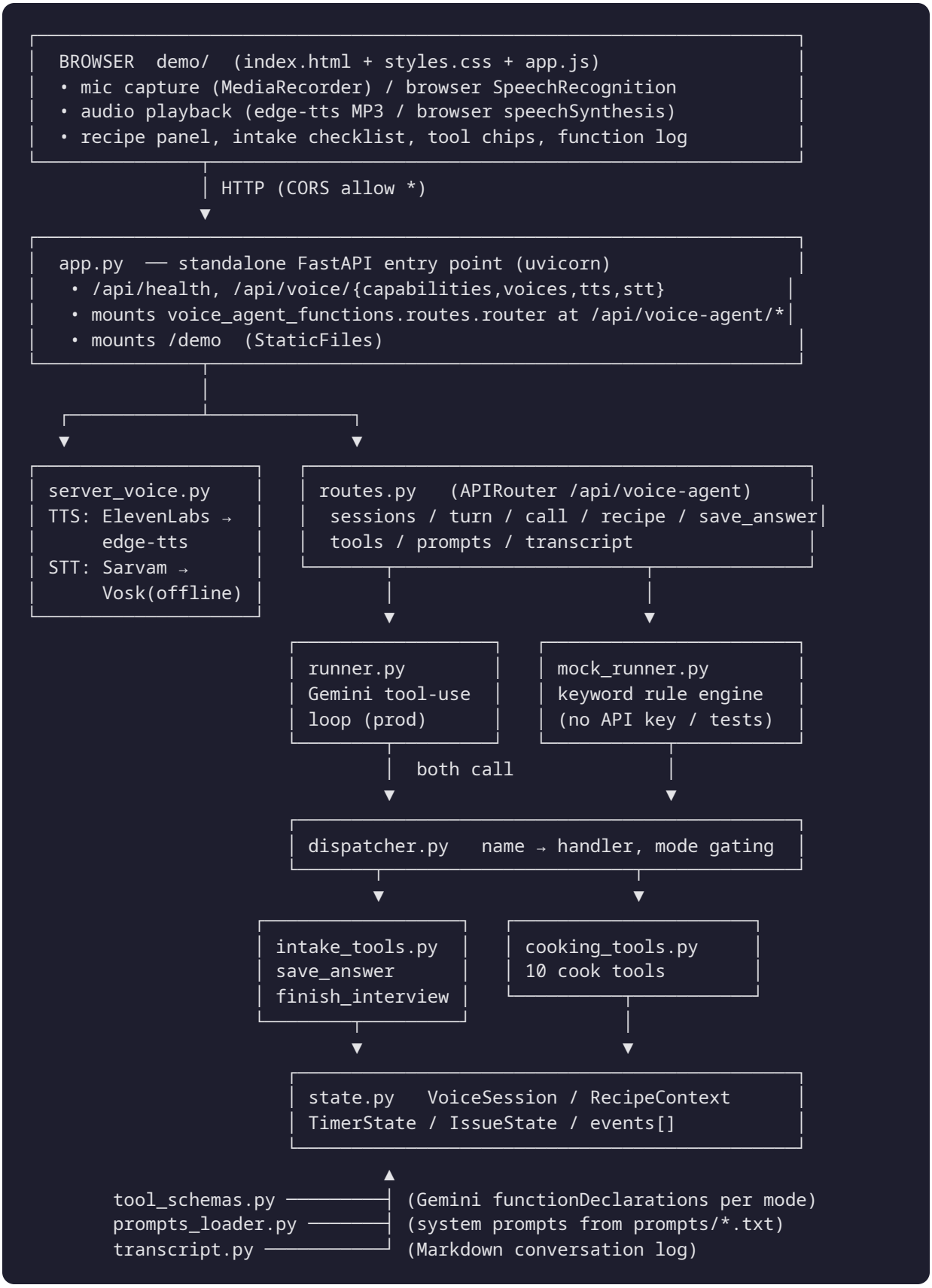
Architecture — `voice_agent_functions`

FastAPI port of the LiveKit voice-agent design (ruchi-voice-agent PR #1). It exposes **12 function-calling tools** over HTTP so a browser/React frontend can drive two conversational agents **without LiveKit**. Works with **zero API keys** (mock runner + free/offline voice).

---

1. Layered view


BROWSER demo/ (index.html + styles.css + app.js)
• mic capture (MediaRecorder) / browser SpeechRecognition
• audio playback (edge-tts MP3 / browser speechSynthesis)
• recipe panel, intake checklist, tool chips, function log
HTTP (CORS allow *)
app.py — standalone FastAPI entry point (uvicorn)
• /api/health, /api/voice/{capabilities,voices,tts,stt}
• mounts voice_agent_functions.routes.router at /api/voice-agent/*
• mounts /demo (StaticFiles)
server_voice.py
TTS: ElevenLabs → edge-tts
STT: Sarvam → Vosk(offline)
routes.py (APIRouter /api/voice-agent)
sessions / turn / call / recipe / save_answer
tools / prompts / transcript
runner.py
Gemini tool-use loop (prod)
mock_runner.py
keyword rule engine (no API key / tests)
both call
dispatcher.py name → handler, mode gating
intake_tools.py
save_answer
finish_interview
cooking_tools.py
10 cook tools
state.py VoiceSession / RecipeContext
TimerState / IssueState / events[]
tool_schemas.py (Gemini functionDeclarations per mode)
prompts_loader.py (system prompts from prompts/*.txt)
transcript.py (Markdown conversation log)

2. Request lifecycle — a single voice turn

```
You speak "తర్వాత ఏంటి?" (what's next?)
       │
       ▼
[app.js] MediaRecorder captures webm → silence detect (1.2s) → stop
       │  POST /api/voice/stt (multipart)
       ▼
[app.py voice_stt]  Sarvam? → Vosk (ffmpeg→16k mono wav) → browser fallback
       │  returns {"transcript": "..."}
       ▼
[app.js sendMessage]  POST /api/voice-agent/sessions/{sid}/turn {text}
       │
       ▼
[routes.run_turn]  GOOGLE_API_KEY set?
       │
       ├── NO  → mock_runner.run_mock_turn
       │            └ regex/Telugu keyword match → dispatcher.dispatch()
       │
       └── YES → runner.GeminiFunctionCallingRunner.run_turn
                    └ loop: generateContent → functionCall → dispatcher
       │
       ▼
[dispatcher.dispatch]  validates mode + name → handler(session, **args)
       │
       ▼
[intake_tools / cooking_tools]  mutate session, append session.events,
       │                        return event dict
       ▼
TurnResponse { assistant_text, tool_calls, tool_results, events, mock }
       │
       ▼
[app.js] applyEvents() → UI updates instantly; speak(assistant_text)
       │
       └── transcript.log_turn() → transcripts/conversation.md
```

3. How functions are actually called (3 paths)

All three end in the same `dispatcher.dispatch()`:

Path	Entry	Who decides the tool	Used by
LLM-driven	`POST /sessions/{sid}/turn` → `runner.py`	Gemini picks `functionCall` from `tool_schemas.py`	production voice/text
Mock-driven	`POST /sessions/{sid}/turn` (no key) → `mock_runner.py`	regex/keyword rules	dev/demo/tests
Direct	`POST /sessions/{sid}/call` or `/save_answer`	frontend/UI explicitly	tool chips, save button

The Gemini tool-use loop (`runner.py:212`)

```
history = []
loop up to max_tool_iters (5):
    payload = system_prompt(mode) + contents + gemini_tools(mode)
              + tool_config AUTO
    data = POST generativelanguage.googleapis.com/v1beta/
              models/{model}:generateContent
           (model fallback chain: configured → 3.5-flash-lite
            → 3.5-flash → 3.1-flash-lite → flash-latest)

    parts = candidates[0].content.parts
    function_calls = [p.functionCall for p in parts]
    text           = "".join(p.text ...)

    if function_calls:
        history += model turn (echo parts verbatim, incl. thoughtSignature)
        for fc in function_calls:
            result = dispatch_function_call(session, fc)   # handler runs here
            response_parts += functionResponse{id,
                                response:{result|error}}
        history += {"role":"user", parts: response_parts}  # NOT role="function"
        continue                                           # loop again
    else:
        history += model text turn
        return Turn(assistant_text=text)
```

Design points:

- **The model decides; the handler just executes.** `dispatcher.py` does *no* intent classification (README rule 1).
- **Templated-response-first:** every handler returns a `phrase` (verbatim canned line), so no extra LLM round-trip to confirm.
- **Errors are returned, not raised:** `dispatch` returns `{"error": ...}`, which becomes `functionResponse.error` so Gemini can self-correct next turn (`dispatcher.py:99-105`).

---

4. Dispatcher — the hub (`dispatcher.py`)

```
dispatch(session, name, args):
    if name not in known_tool_names(session.mode):  → {"error": out-of-scope}
    handler = _HANDLERS[name]                # name → lambda(session, **kw)
    try:    event = handler(session, **args)
    except TypeError → {"error": bad args}
    except ValueError → {"error": str(exc)}
    return {"result": event, "name": name}
```

`known_tool_names(mode)` is the mode gate:

- `intake` → `{save_answer, finish_interview}`
- `cook` → the 10 cooking tools

`_HANDLERS` maps each of the 12 names to a thin lambda that forwards named args to the real handler (`dispatcher.py:35-54`).

---

5. State model (`state.py`)

```
VoiceSession
├── session_id: str
├── mode: "intake" | "cook"
├── answers: {field: value}          # intake
├── current_field: str               # intake pointer
├── recipe: RecipeContext | None     # cook
│      ├── recipe_name
│      ├── steps: list[str]
│      ├── step_index: int
│      └── step_text() / from_metadata()
├── paused: bool
├── active_timer: TimerState | None  # {seconds, label, asyncio.Task|Timer}
├── active_issue: IssueState | None  # {issue, details}
└── events: list[dict]               # append-only event log UI replays
```

`INTAKE_FIELDS` order: `name → time → servings → level → ingredients → utensils → tips(optional) → steps(last)` . Sessions live in-memory in `SessionManager` (`runner.py:98`), keyed by id.

---

6. The 12 tools

Intake (`intake_tools.py`)

```
save_answer(field, value)
   ├─ validate field ∈ INTAKE_FIELDS
   ├─ enforce field == session.current_field   (refuse to skip ahead)
   ├─ normalize tips ("skip"/"no"/"") → ""
   ├─ store answer, advance pointer
   └─ if last field → finish_interview()
finish_interview()
   └─ emit interview_complete{answers: all 8, tips always present};
      current_field=""
```

Cooking (`cooking_tools.py`)

```
next_step / previous_step / goto_step(step)
   └─ goto_step() clamps index; out-of-range → no_op + canned phrase
start_timer(seconds, label)
   └─ asyncio.create_task (or threading.Timer w/o event loop)
      on expiry appends timer_expired to session.events (server-driven)
cancel_timer / pause / resume / report_issue(issue, details) / resolve_issue
finish_cooking → cancels timer, emits cooking_finished
```

`report_issue` validates against fixed enum `{spicy, salty, watery, burnt, bland, other}`.

Every handler: **validate → mutate session → append `events[]` → return event dict with `phrase` / `lead_in`**.

---

7. Voice I/O fallback chains (`server_voice.py` + `app.py`)

```
TTS  POST /api/voice/tts
  1. ElevenLabs  (ELEVEN_API_KEY + ELEVEN_VOICE_ID, eleven_v3_conversational)
  2. edge-tts    (free, te-IN-ShrutiNeural)              → audio/mpeg
  3. browser speechSynthesis                             → JSON {"fallback":"browse

STT  POST /api/voice/stt
  1. Sarvam      (SARVAM_API_KEY, saaras:v3, te-IN)
  2. Vosk        (offline; ffmpeg converts webm→16kHz mono WAV)
  3. browser SpeechRecognition

LLM  /turn
  1. Gemini function-calling (GOOGLE_API_KEY)
  2. mock_runner (keyword rules)
```

`GET /api/voice/capabilities` reports which are live; the demo shows this in a startup bubble.

---

8. HTTP API surface

```
/api/voice-agent
  POST   /sessions                 create {mode, recipe?}
  GET    /sessions/{sid}           state + events
  DELETE /sessions/{sid}           drop (cancels timer)
  PUT    /sessions/{sid}/recipe    swap recipe mid-session
  POST   /sessions/{sid}/turn      one utterance → LLM/mock loop
  POST   /sessions/{sid}/call      fire a tool directly (bypass LLM)
  POST   /sessions/{sid}/save_answer
  GET    /tools?mode=intake|cook   Gemini functionDeclarations
  GET    /prompts                  raw system prompts
  GET/DELETE /transcript           read / clear Markdown log
  GET    /transcript/files         list files

/api/voice   capabilities · voices · tts · stt
/api/health · / · /demo/ · /docs
```

---

9. File responsibility map

File	Role
`app.py`	FastAPI app: voice endpoints, CORS, mounts router + demo
`voice_agent_functions/routes.py`	`/api/voice-agent/*` router, request models, session manager singleton
`runner.py`	Gemini tool-use loop + `SessionManager` (production path)
`mock_runner.py`	Regex/Telugu keyword router (no-key path)
`dispatcher.py`	name→handler routing, mode gating, error envelopes
`intake_tools.py` / `cooking_tools.py`	The 12 handlers + canned phrases
`state.py`	`VoiceSession`, `RecipeContext`, `INTAKE_FIELDS`, timer/issue dataclasses
`tool_schemas.py`	Gemini `functionDeclarations` per mode
`prompts_loader.py` + `prompts/*.txt`	system prompts (intake, cooking base + overlay)
`server_voice.py`	ElevenLabs/edge-tts + Sarvam/Vosk adapters, capability probe
`transcript.py`	append-only Markdown conversation log
`demo/`	standalone browser client (orb, chips, recipe/intake panels, log)
`tests/test_tools.py`	47 unit tests over dispatcher/handlers

---

10. Why it's built this way

1. **Transport-agnostic core** — `runner.py` takes/returns plain text; no LiveKit/audio coupling, so it can sit behind HTTP, a websocket, or a CLI.
2. **Deterministic, testable tools** — dispatch logic is pure Python, unit-testable without an LLM.
3. **Graceful degradation everywhere** — every capability (LLM, TTS, STT) has a fallback, so the demo runs fully offline with zero keys.

4. **Event-sourced UI** — handlers append to `session.events`; the frontend replays new events for instant updates (with a 5s poll as safety net).